DR. RVR NRI DEEMED TO BE UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING (ABL) ASSIGNMENT

Student Name	O. Jagadeesh Babu	Roll Number	26000048
Course	Python Programming	Academic Year	2026–2027
Branch & Section	CSE	Experiments	1–10

CASE STUDY INDEX

No.	Case Study Title	Main Concepts Covered
1	Campus Resource Allocation Under Constraints	Nested lists, tuple unpacking, dictionaries, loops, conditional allocation, shortage calculation.
2	Student Academic Risk Classification	Lists, parallel data, nested conditions, list accumulation, classification.
3	Retail Order Dispatch and Return Analysis	Dictionaries, loops, arithmetic, inventory updates, validation.
4	Energy Conservation Before-and-After Analysis	Dictionaries, arithmetic, percentage calculations, sorting.
5	Nested List Sports Performance Analysis	Nested lists, loops, sum, mean, range, sorting with tie-breakers.
6	Multi-Stage Scholarship Shortlisting	Tuples, multi-condition filtering, loops, sorting, quota selection.
7	Integrated University Fee and Eligibility Audit	Dictionaries, lists, multiple conditions, reason tracking, reporting.
8	Smart Hostel Resource Monitoring	Nested dictionaries, loops, arithmetic, threshold checks, aggregation.
9	Multi-Route Courier Dispatch Planning	Lists of tuples, capacity tracking, conditional allocation, loops.
10	Campus Festival Registration and Financial Audit	Lists, sets, duplicate detection, arithmetic, financial summaries.

Each experiment includes its aim, concepts, algorithm, Python source code, test conditions, and sample output.

Experiment 1: Campus Resource Allocation Under Constraints

Student: O. Jagadeesh Babu Roll No.: 26000048

AIM

Compare equipment requirements with available resources across multiple activities, calculate shortages, and allocate available equipment in a specified priority order.

CONCEPTS

Nested lists, tuple unpacking, dictionaries, loops, conditional allocation, shortage calculation.

ALGORITHM & LOGIC	TEST CASES / CONDITIONS
1. Store each activity with its priority and required equipment quantity. 2. Store the available equipment inventory. 3. Process activities in priority order, allocating only the quantity available. 4. Calculate each activity's shortage and update the remaining inventory. 5. Display allocations, shortages, and final inventory.	Seminar requires 10 projectors; Sports requires 8 kits; Cultural requires 7 sound systems. Only 2 sound systems are available.

PYTHON SOURCE CODE

SAMPLE EXECUTION OUTPUT

```
Seminar Allocated: 10
Shortage: 0
Sports Allocated: 8
Shortage: 0
Cultural Allocated: 2
Shortage: 5
Remaining equipment: 0
```

Experiment 2: Student Academic Risk Classification

Student: O. Jagadeesh Babu Roll No.: 26000048

AIM

Combine marks, attendance, and laboratory scores to classify students into risk categories, identify simultaneous risk factors, and generate a summary.

CONCEPTS

Lists, parallel data, nested conditions, list accumulation, classification.

ALGORITHM & LOGIC	TEST CASES / CONDITIONS
1. Store student names, marks, attendance, and lab scores in corresponding lists. 2. Check each student's marks, attendance, and lab score against selected thresholds. 3. Collect every risk factor that applies to a student. 4. Classify students as High, Medium, or Low risk based on the number of factors. 5. Print each student's category and a reason-wise summary.	Thresholds: marks below 50, attendance below 75%, and laboratory score below 50. Two or more risk factors indicate High Risk; one indicates Medium Risk.

PYTHON SOURCE CODE

SAMPLE EXECUTION OUTPUT

```
Ravi High Risk ['Low marks', 'Low attendance', 'Low lab score']
Sita Low Risk []
Raju Medium Risk ['Low attendance']
```

Experiment 3: Retail Order Dispatch and Return Analysis

Student: O. Jagadeesh Babu Roll No.: 26000048

AIM

Process order quantities, dispatch records, and returned quantities; calculate outstanding orders, update inventory, and detect inconsistent records.

CONCEPTS

Dictionaries, loops, arithmetic, inventory updates, validation.

ALGORITHM & LOGIC	TEST CASES / CONDITIONS
1. Store each order's ordered quantity, dispatched quantity, and returned quantity. 2. Check that dispatched quantity does not exceed ordered quantity and returns do not exceed dispatches. 3. Calculate outstanding quantity as ordered minus dispatched. 4. Update inventory by adding returned items back to stock. 5. Report order-wise values, inconsistencies, and final inventory.	Orders: (50, 20, 5), (25, 25, 10), (40, 30, 5) as ordered, dispatched, returned. Initial inventory is 20.

PYTHON SOURCE CODE

SAMPLE EXECUTION OUTPUT

```
Order: 1
Pending: 30
Returned: 5
Order: 2
Pending: 0
Returned: 10
Order: 3
Pending: 10
Returned: 5
Final inventory: 40
```

Experiment 4: Energy Conservation Before-and-After Analysis

Student: O. Jagadeesh Babu Roll No.: 26000048

AIM

Compare appliance-wise electricity usage before and after an intervention, calculate savings percentages, and rank appliances by savings.

CONCEPTS

Dictionaries, arithmetic, percentage calculations, sorting.

ALGORITHM & LOGIC	TEST CASES / CONDITIONS
1. Store before- and after-intervention usage for each appliance. 2. Calculate energy saved by subtracting after usage from before usage. 3. Calculate savings percentage relative to before usage. 4. Sort appliances by savings percentage in descending order. 5. Print appliance-wise savings and ranking.	Fan: 100 to 70 units; Light: 80 to 40 units; AC: 300 to 240 units.

PYTHON SOURCE CODE

SAMPLE EXECUTION OUTPUT

```
Fan Saved: 30
Savings percentage: 30.0
Light Saved: 40
Savings percentage: 50.0
AC Saved: 60
Savings percentage: 20.0
Ranking:
Light 50.0
Fan 30.0
AC 20.0
```

Experiment 5: Nested List Sports Performance Analysis

Student: O. Jagadeesh Babu Roll No.: 26000048

AIM

Analyse players' scores across multiple matches, calculate consistency measures, identify trends, and rank players using supplied tiebreaking rules.

CONCEPTS

Nested lists, loops, sum, mean, range, sorting with tie-breakers.

ALGORITHM & LOGIC	TEST CASES / CONDITIONS
1. Store each player's scores across several matches in a nested list. 2. Calculate average score and score range for each player. 3. Determine the trend by comparing the first and last match scores. 4. Rank players by average score, then by smaller score range as a consistency tiebreaker. 5. Display statistics, trends, and final rankings.	Ravi: 90, 85, 95; Sita: 80, 82, 84; Raju: 75, 90, 85. Ranking uses higher average first, then smaller score range.

PYTHON SOURCE CODE

SAMPLE EXECUTION OUTPUT

```
Ravi 270 90.0 Improving
Sita 246 82.0 Improving
Raju 250 83.3 Improving
Rankings:
('Ravi', 90.0)
('Raju', 83.3)
('Sita', 82.0)
```

Experiment 6: Multi-Stage Scholarship Shortlisting

Student: O. Jagadeesh Babu Roll No.: 26000048

AIM

Apply minimum academic eligibility, attendance requirements, income conditions, and ranking rules to select candidates within a fixed scholarship quota.

CONCEPTS

Tuples, multi-condition filtering, loops, sorting, quota selection.

ALGORITHM & LOGIC	TEST CASES / CONDITIONS
1. Store candidate name, marks, attendance, income, and subject marks. 2. Apply all eligibility conditions. 3. Sort eligible candidates by marks in descending order. 4. Select candidates only up to the fixed scholarship quota. 5. Print selected candidates and candidates not selected because of quota or ineligibility.	Eligibility: marks ≥ 75, attendance ≥ 75%, income < ₹250,000, subject marks > 40. Quota: 2 candidates.

PYTHON SOURCE CODE

SAMPLE EXECUTION OUTPUT

```
Selected candidates:
('Sita', 95)
('Raju', 90)
Not selected / ineligible:
Ravi
Latha
```

Experiment 7: Integrated University Fee and Eligibility Audit

Student: O. Jagadeesh Babu Roll No.: 26000048

AIM

Combine fee payments, attendance, internal marks, and examination eligibility records; identify outstanding conditions and produce a reason-wise report.

CONCEPTS

Dictionaries, lists, multiple conditions, reason tracking, reporting.

ALGORITHM & LOGIC	TEST CASES / CONDITIONS
1. Store each student's fee due, fee paid, attendance, and internal marks. 2. Compare paid and due fees to identify outstanding balances. 3. Check attendance and internal marks against eligibility thresholds. 4. Build a list of all reasons each student has outstanding conditions. 5. Print eligibility status, outstanding fee, and reason-wise conditions.	Eligibility thresholds: fees fully paid, attendance ≥ 75%, and internal marks ≥ 35.

PYTHON SOURCE CODE

SAMPLE EXECUTION OUTPUT

```
Ravi Eligible
Outstanding fee: 0
Conditions: []
Sita Not eligible
Outstanding fee: 10000
Conditions: ['Fee pending', 'Low attendance']
Raju Not eligible
Outstanding fee: 0
Conditions: ['Low internal marks']
```

Experiment 8: Smart Hostel Resource Monitoring

Student: O. Jagadeesh Babu Roll No.: 26000048

AIM

Analyse room-wise electricity and water consumption, calculate charges, identify threshold violations, and compare resource utilization across floors.

CONCEPTS

Nested dictionaries, loops, arithmetic, threshold checks, aggregation.

ALGORITHM & LOGIC	TEST CASES / CONDITIONS
1. Store room number, floor, electricity usage, and water usage. 2. Calculate electricity and water charges using fixed unit rates. 3. Calculate each room's total charge. 4. Compare consumption with defined thresholds and issue warnings. 5. Aggregate and compare consumption by floor.	Electricity rate ₹5/unit; water rate ₹25/unit. High consumption warning if electricity > 200 units or water > 10 units.

PYTHON SOURCE CODE

SAMPLE EXECUTION OUTPUT

```
Room: 101 Floor: 1
Total charge: 625.0
Normal consumption
Room: 102 Floor: 1
Total charge: 950.0
Normal consumption
Room: 201 Floor: 2
Total charge: 500.0
Normal consumption
Room: 202 Floor: 2
Total charge: 1550.0
Warning: High consumption
Floor-wise total usage:
Floor 1: 263
Floor 2: 346
```

Experiment 9: Multi-Route Courier Dispatch Planning

Student: O. Jagadeesh Babu Roll No.: 26000048

AIM

Process package weights, destination zones, vehicle capacities, and delivery priorities; allocate packages to vehicles and identify packages that cannot be dispatched.

CONCEPTS

Lists of tuples, capacity tracking, conditional allocation, loops.

ALGORITHM & LOGIC	TEST CASES / CONDITIONS
1. Store vehicles with route zone and available capacity. 2. Store packages with ID, destination zone, weight, and priority. 3. Process packages in descending priority order. 4. Assign a package only when route matches and remaining vehicle capacity is sufficient. 5. Report dispatched packages, rejected packages, and remaining capacities.	Vehicle1 serves zone A with capacity 15; Vehicle2 serves zone B with capacity 10. Package records contain ID, zone, weight, and priority.

PYTHON SOURCE CODE

SAMPLE EXECUTION OUTPUT

```
Package 5 sent using Vehicle1
Package 8 sent using Vehicle2
Package 4 cannot be dispatched
Package 12 cannot be dispatched
Remaining capacities:
['Vehicle1', 'A', 10]
['Vehicle2', 'B', 2]
```

Experiment 10: Campus Festival Registration and Financial Audit

Student: O. Jagadeesh Babu Roll No.: 26000048

AIM

Reconcile participant lists, registration fees, sponsorship contributions, and expenses; detect duplicate registrations and calculate the final financial balance.

CONCEPTS

Lists, sets, duplicate detection, arithmetic, financial summaries.

ALGORITHM & LOGIC	TEST CASES / CONDITIONS
1. Read the participant registration list and identify duplicate names. 2. Keep one valid registration per participant. 3. Calculate registration income from valid participants. 4. Add sponsorship contributions and subtract expenses. 5. Print valid participants, duplicate records, and final balance.	Registrations: Ravi, Sita, Raju, Ravi; fee ₹500 per valid participant; sponsorship ₹8,000; expenses ₹4,500.

PYTHON SOURCE CODE

SAMPLE EXECUTION OUTPUT

```
Duplicate registration: Ravi
Valid participants: ['Ravi', 'Sita', 'Raju']
Registration fees: 1500
Sponsorships: 8000
Expenses: 4500
Final balance: 5000
```